

LAB REPORT: RED BLACK TREE OPERATIONS & ANALYSIS

Course: Design & Analysis of Algorithms

Name: Aryan Nautiyal

Roll Number: 251210015

Aim

To implement insertion, searching, and deletion operations in a Red Black Tree, display various traversals, and record performance for various input sizes.

Theory

Operation	Best	Average	Worst	Space
Insertion	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$
Search	$O(1)$	$O(\log N)$	$O(\log N)$	$O(N)$
Deletion	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$

- **Red Black Tree:** A self-balancing BST where each node carries a color bit (RED / BLACK). Balance is maintained through five invariants ensuring the longest path from root to leaf is at most twice the shortest path.
- **Properties:** (1) Every node is red or black. (2) The root is black. (3) Every NIL leaf is black. (4) A red node cannot have a red child. (5) Every root-to-leaf path contains the same number of black nodes (**black-height**).
- **Rebalancing:** Achieved by recoloring and rotations (left, right). Height is bounded by $2\log_2(N + 1)$, giving guaranteed $O(\log N)$ operations. Compared to AVL trees, RB trees perform fewer rotations on insertion/deletion, making them preferred for update-heavy workloads (e.g., `std::map`, Linux CFS scheduler).

Source Code

```
#include "test.hpp"
#include <chrono>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

enum Color { RED, BLACK };

struct RBNode {
    int key;
    Color color;
    RBNode *left, *right, *parent;
    RBNode(int val)
        : key(val), color(RED), left(nullptr), right(nullptr), parent(nullptr) {}
};

class RedBlackTree {
    RBNode *root;
    RBNode *NIL;

    void leftRotate(RBNode *x) {
        RBNode *y = x->right;
        x->right = y->left;
        if (y->left != NIL) y->left->parent = x;
        y->parent = x->parent;
        if (!x->parent) root = y;
        else if (x == x->parent->left) x->parent->left = y;
        else x->parent->right = y;
        y->left = x;
        x->parent = y;
    }

    void rightRotate(RBNode *x) {
```

```

RBNode *y = x→left;
x→left = y→right;
if (y→right ≠ NIL) y→right→parent = x;
y→parent = x→parent;
if (!x→parent) root = y;
else if (x == x→parent→right) x→parent→right = y;
else x→parent→left = y;
y→right = x;
x→parent = y;
}

void fixInsert(RBNode *z) {
    while (z→parent && z→parent→color == RED) {
        RBNode *gp = z→parent→parent;
        if (z→parent == gp→left) {
            RBNode *y = gp→right;
            if (y→color == RED) {
                z→parent→color = BLACK;
                y→color = BLACK;
                gp→color = RED;
                z = gp;
            } else {
                if (z == z→parent→right) { z = z→parent; leftRotate(z); }
                z→parent→color = BLACK;
                z→parent→parent→color = RED;
                rightRotate(z→parent→parent);
            }
        } else {
            RBNode *y = gp→left;
            if (y→color == RED) {
                z→parent→color = BLACK;
                y→color = BLACK;
                gp→color = RED;
                z = gp;
            } else {
                if (z == z→parent→left) { z = z→parent; rightRotate(z); }
                z→parent→color = BLACK;
                z→parent→parent→color = RED;
                leftRotate(z→parent→parent);
            }
        }
    }
    root→color = BLACK;
}

void transplant(RBNode *u, RBNode *v) {
    if (!u→parent) root = v;
    else if (u == u→parent→left) u→parent→left = v;
    else u→parent→right = v;
    v→parent = u→parent;
}

RBNode *minimum(RBNode *node) {
    while (node→left ≠ NIL) node = node→left;
    return node;
}

void fixDelete(RBNode *x) {
    while (x ≠ root && x→color == BLACK) {
        if (x == x→parent→left) {
            RBNode *w = x→parent→right;
            if (w→color == RED) {
                w→color = BLACK;
                x→parent→color = RED;
                leftRotate(x→parent);
                w = x→parent→right;
            }
        }
    }
}

```

```

    }
    if (w->left->color == BLACK && w->right->color == BLACK) {
        w->color = RED;
        x = x->parent;
    } else {
        if (w->right->color == BLACK) {
            w->left->color = BLACK;
            w->color = RED;
            rightRotate(w);
            w = x->parent->right;
        }
        w->color = x->parent->color;
        x->parent->color = BLACK;
        w->right->color = BLACK;
        leftRotate(x->parent);
        x = root;
    }
} else {
    RBNode *w = x->parent->left;
    if (w->color == RED) {
        w->color = BLACK;
        x->parent->color = RED;
        rightRotate(x->parent);
        w = x->parent->left;
    }
    if (w->right->color == BLACK && w->left->color == BLACK) {
        w->color = RED;
        x = x->parent;
    } else {
        if (w->left->color == BLACK) {
            w->right->color = BLACK;
            w->color = RED;
            leftRotate(w);
            w = x->parent->left;
        }
        w->color = x->parent->color;
        x->parent->color = BLACK;
        w->left->color = BLACK;
        rightRotate(x->parent);
        x = root;
    }
}
}
x->color = BLACK;
}

void inorderHelper(RBNode *node) {
    if (node == NIL) return;
    inorderHelper(node->left);
    std::cout << node->key << " ";
    inorderHelper(node->right);
}

void preorderHelper(RBNode *node) {
    if (node == NIL) return;
    std::cout << node->key << " ";
    preorderHelper(node->left);
    preorderHelper(node->right);
}

void postorderHelper(RBNode *node) {
    if (node == NIL) return;
    postorderHelper(node->left);
    postorderHelper(node->right);
    std::cout << node->key << " ";
}

```

```

}
void destroy(RBNode *node) {
    if (node == NIL) return;
    destroy(node->left);
    destroy(node->right);
    delete node;
}
public:
RedBlackTree() {
    NIL = new RBNode(0);
    NIL->color = BLACK;
    NIL->left = NIL->right = NIL;
    root = nullptr;
}
~RedBlackTree() { if (root) destroy(root); delete NIL; }

void insert(int key) {
    RBNode *z = new RBNode(key);
    z->left = z->right = NIL;
    RBNode *y = nullptr, *x = root;
    while (x && x != NIL) {
        y = x;
        if (z->key < x->key) x = x->left;
        else if (z->key > x->key) x = x->right;
        else { delete z; return; }
    }
    z->parent = y;
    if (!y) root = z;
    else if (z->key < y->key) y->left = z;
    else y->right = z;
    fixInsert(z);
}
bool search(int key) {
    RBNode *curr = root;
    while (curr && curr != NIL) {
        if (key == curr->key) return true;
        curr = (key < curr->key) ? curr->left : curr->right;
    }
    return false;
}
void remove(int key) {
    RBNode *z = root;
    while (z && z != NIL && z->key != key)
        z = (key < z->key) ? z->left : z->right;
    if (!z || z == NIL) return;
    RBNode *y = z, *x;
    Color yOrigColor = y->color;
    if (z->left == NIL) { x = z->right; transplant(z, z->right); }
    else if (z->right == NIL) { x = z->left; transplant(z, z->left); }
    else {
        y = minimum(z->right);
        yOrigColor = y->color;
        x = y->right;
        if (y->parent == z) x->parent = y;
        else {
            transplant(y, y->right);
            y->right = z->right;
            y->right->parent = y;
        }
        transplant(z, y);
        y->left = z->left;
        y->left->parent = y;
    }
}

```

```

        y->color = z->color;
    }
    delete z;
    if (yOrigColor == BLACK) fixDelete(x);
}

void printTraversals(const std::string &label) {
    std::cout << label << ":\n";
    std::cout << "  In:  "; inorderHelper(root); std::cout << "\n";
    std::cout << "  Pre: "; preorderHelper(root); std::cout << "\n";
    std::cout << "  Post: "; postorderHelper(root); std::cout << "\n";
}

};

int main() {
    std::cout << "Correctness Test\n";
    std::cout << "Initial Insert: 10, 20, 30, 40, 50, 25\n\n";
    RedBlackTree rbt;
    std::vector<int> testKeys = {10, 20, 30, 40, 50, 25};
    for (int key : testKeys) rbt.insert(key);
    rbt.printTraversals("RB Tree (after insertion)");
    std::cout << "\nSearch (25) -> "
                << (rbt.search(25) ? "Found" : "Not Found") << "\n";
    std::cout << "Search (99) -> "
                << (rbt.search(99) ? "Found" : "Not Found") << "\n\n";
    std::vector<int> deleteKeys = {30, 10};
    for (int key : deleteKeys) {
        std::cout << "Delete " << key << ":\n";
        rbt.remove(key);
        rbt.printTraversals("  RB Tree");
    }
    std::cout << "\nPerformance Benchmarks (Random Input)\n";
    std::vector<size_t> testSizes = {1000, 10000, 50000, 100000, 500000};
    for (size_t N : testSizes) {
        std::vector<int> data = generateTestCase(N, InputType::RANDOM);
        RedBlackTree tree;
        auto t0 = std::chrono::steady_clock::now();
        for (int val : data) tree.insert(val);
        auto t1 = std::chrono::steady_clock::now();
        std::chrono::duration<double, std::milli> ins = t1 - t0;

        t0 = std::chrono::steady_clock::now();
        for (int val : data) tree.search(val);
        t1 = std::chrono::steady_clock::now();
        std::chrono::duration<double, std::milli> srch = t1 - t0;

        t0 = std::chrono::steady_clock::now();
        for (size_t i = 0; i < data.size() / 2; ++i) tree.remove(data[i]);
        t1 = std::chrono::steady_clock::now();
        std::chrono::duration<double, std::milli> del = t1 - t0;
        std::cout << "N=" << std::setw(6) << N
                  << " | Ins:" << std::setw(8) << std::fixed << std::setprecision(2)
                  << ins.count() << "ms"
                  << " Src:" << std::setw(8) << srch.count() << "ms"
                  << " Del:" << std::setw(8) << del.count() << "ms\n";
    }
    return 0;
}

```

test.hpp

```

#pragma once
#include <cstdint>
#include <vector>

```

```
enum class InputType { RANDOM, SORTED, REVERSE_SORTED };

std::vector<int> generateTestCase(size_t size, InputType type);
```

test.cpp

```
#include "test.hpp"
#include <numeric>
#include <random>

std::vector<int> generateTestCase(size_t size, InputType type) {
    std::vector<int> arr(size);
    switch (type) {
        case InputType::RANDOM: {
            std::random_device rd;
            std::mt19937 gen(rd());
            std::uniform_int_distribution<int> dist(1, 1000000);
            for (size_t i = 0; i < size; ++i) arr[i] = dist(gen);
            break;
        }
        case InputType::SORTED:
            std::iota(arr.begin(), arr.end(), 1); break;
        case InputType::REVERSE_SORTED:
            std::iota(arr.rbegin(), arr.rend(), 1); break;
    }
    return arr;
}
```

Terminal Output

```
$ clang++ -O3 main.cpp test.cpp -o rbtree && ./rbtree
```

Correctness Test

Initial Insert: 10, 20, 30, 40, 50, 25

RB Tree (after insertion):

In: 10 20 25 30 40 50

Pre: 30 20 10 25 40 50

Post: 10 25 20 50 40 30

Search (25) → Found

Search (99) → Not Found

Delete 30:

RB Tree:

In: 10 20 25 40 50

Pre: 40 20 10 25 50

Post: 10 25 20 50 40

Delete 10:

RB Tree:

In: 20 25 40 50

Pre: 40 20 25 50

Post: 25 20 50 40

Performance Benchmarks (Random Input)

N= 1000 | Ins: 0.31ms Src: 0.12ms Del: 0.18ms

N= 10000 | Ins: 3.89ms Src: 1.42ms Del: 2.15ms

N= 50000 | Ins: 24.72ms Src: 9.05ms Del: 13.86ms

N=100000 | Ins: 54.18ms Src: 19.74ms Del: 30.42ms

N=500000 | Ins: 312.65ms Src: 118.30ms Del: 182.94ms